

回溯法

前言

编程学到现在才真正到了戏肉部分，从这里往下学，你才知道什么叫做博大精深。今天我们要啃的这块硬骨头叫做深度优先搜索法(只需掌握回溯法即可)。

首先我们来想象一只老鼠，在一座不见天日的迷宫内，老鼠在入口处进去，要从出口出来。那老鼠会怎么走？当然是这样的：老鼠如果遇到直路，就一直往前走，如果遇到分叉路口，就任意选择其中的一个继续往下走，如果遇到死胡同，就退回到最近的一个分叉路口，选择另一条道路再走下去，如果遇到出口，老鼠的旅途就算结束了。**深度优先搜索法的基本原则**就是这样：按照某种条件往前试探搜索，如果前进中遭到失败（正如老鼠遇到死胡同）则退回头另选通路继续搜索，直到找到条件的目标为止。

实现这一算法，我们要用到编程的另一大利器——递归。递归是一个很抽象的概念，但是在日常生活中，我们还是能够看到的。拿两面镜子来，把他们面对着面，你会看到什么？你会看到镜子中有无数个镜子？怎么回事？A 镜子中有 B 镜子的象，B 镜子中有 A 镜子的象，A 镜子的象就是 A 镜子本身的真实写照，也就是说 A 镜子的象包括了 A 镜子，还有 B 镜子在 A 镜子中的象……好累啊，又烦又绕口，还不好理解。换成计算机语言就是 A 调用 B，而 B 又调用 A，这样间接的，A 就调用了 A 本身，这实现了一个重复的功能。

再举一个例子：从前有座山，山里有座庙，庙里有个老和尚，老和尚在讲故事，讲什么呢？讲：从前有座山，山里有座庙，庙里有个老和尚，老和尚在讲故事，讲什么呢？讲：从前有座山，山里有座庙，庙里有个老和尚，老和尚在讲故事，讲什么呢？讲：……。好家伙，这样讲到世界末日还讲不玩，老和尚讲的故事实际上就是前面的故事情节，这样不断地调用程序本身，就形成了递归。万一这个故事中的某一个老和尚看这个故事不顺眼，就把他要讲的故事换成：“你有完没完啊！”，这样，整个故事也就嘎然而止了。我们编程就要注意这一点，在适当的时候，就必须要有个这样的和尚挺身而出，把整个故事给停下来，或者使他不再往深一层次搜索，要不，我们的递归就会因计算机存储容量的限制而被迫溢出，切记，切记。

★ 递归策略：

一个过程或函数在其定义或说明中又直接或间接调用自身的一种方法，它通常把一个大型复杂的问题层层转化为一个与原问题相似的规模较小的问题来求解，递归策略只需少量的程序就可描述出解题过程所需要的多次重复计算，大大地减少了程序的代码量。

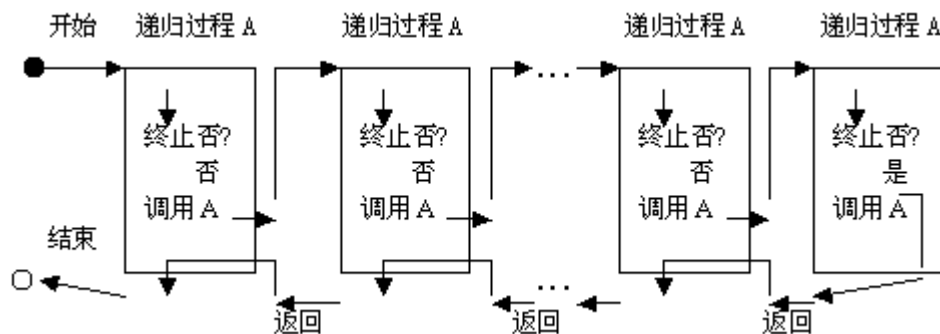
注意：

(1) 递归就是在过程或函数里调用自身；

(2) 在使用递归策略时，必须有一个明确的递归结束条件，称为递归出口。

递归过程的执行总是一个过程体未执行完，就带着本次执行的结果又进入另一轮过程体的执行，……，如此反复，不断深入，直到某次过程的执行遇到终止递归调用的条件成立时，则不再深入，而执行本次的过程体余下的部分，然后又返回到上一次调用的过程体中，执行其余下的部分，……，如此反复，直到回到起始位置上，才最终结束整个递归过程的执行，得到相应的执行结果。递归过程的程序设计的核心就是参照这种执行流程，设计出一种适合“逐步深入，而后又逐步返回”的递归调用模型，以解决实际问题。

递归算法作为计算机程序设计中的一种重要的算法，是较难理解的算法之一。简单地说，递归就是编写这样的一个特殊的过程，该过程体中有一个语句用于调用过程自身（称为递归调用）。递归过程由于实现了自我的嵌套执行，使这种过程的执行变得复杂起来，其执行的流程可以用图 1 所示。



(背住)：注意递归过程间可看作是子程序名相同的不同模块，它们之间有父子关系，子程序在执行结束只返回到它的父亲模块。

回溯是按照某种条件往前试探搜索,若前进中遭到失败,则回过头来另择通路继续搜索.

例 15: N 皇后问题

在 $N \times N$ 的棋盘上放置 N 个皇后而彼此不受攻击 (即在棋盘的任一行,任一列和任一对角线上不能放置 2 个皇后),编程求解所有的摆放方法。

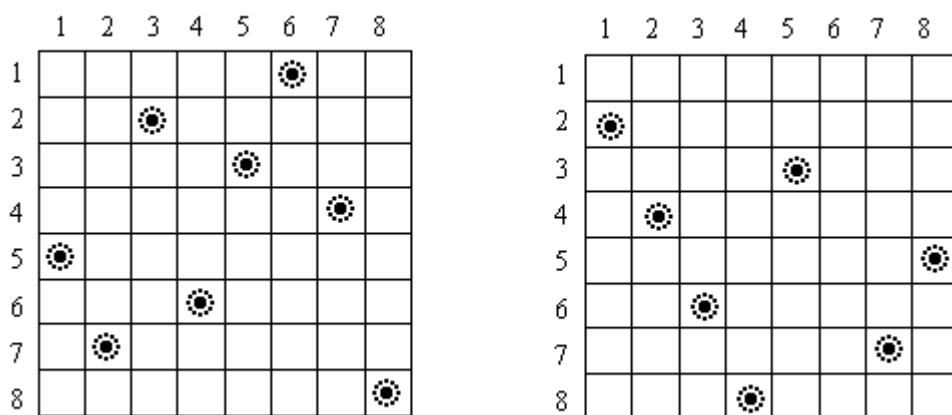


图 14 八皇后的两组解

分析:

这是一道很经典的题目了,我们先要明确一下思路,如何运用深度优先搜索法,完成这道题目。我们先建立一个 8×8 格的棋盘,在棋盘的第一行的任意位置安放一只皇后。紧接着,我们就来放 第二行,第二行的安放就要受一些限制了,因为与第一行的皇后在同一竖行或同一对角线的位置上是不能安放皇后的,接下来是第三行,……,或许我们会遇到这种情况,在摆到某一行时,无论皇后摆放在什么位置,她都会被其他行的皇后吃掉,这说明什么呢?这说明,我们前面的摆放是失败的,也就是说,按照前面的皇后的摆放方法,我们不可能得到正确的解。那这时怎么办?改啊,我们回到上一行,把原先我们摆好的皇后换另外一个位置,接着再回过头摆这一行,如果这样还不行或者上一行的皇后只有一个位置可放,那怎么办?我们回到上一行的上一行,这和老鼠碰了壁就回头是一个意思。就这样的不断的尝试,修正,尝试修正,我们最终会得到正确的结论的。

由于皇后的摆放位置不能通过某种公式来确定,因此对于每个皇后的摆放位置都要进行试探和纠正,这就是“回溯”的思想。在 N 个皇后未放置完成前,摆放第 I 个皇后和第 $I+1$ 个皇后的试探方法是相同的,因此完全可以采用递归的方法来处理。

下面是放置第 I 个皇后的递归算法,也是所有回溯法程序的通用框架:

Procedure Try(I :integer);

{搜索第 I 行皇后的位置}

var

j :integer;

begin

 if $I=n+1$ then 输出方案;

 for $j:=1$ to n do

 if 皇后能放在第 I 行第 J 列的位置 then begin

 放置第 I 个皇后;

 对放置皇后的位置进行标记;

 Try ($I+1$)

 对放置皇后的位置释放标记;

 End;

End;

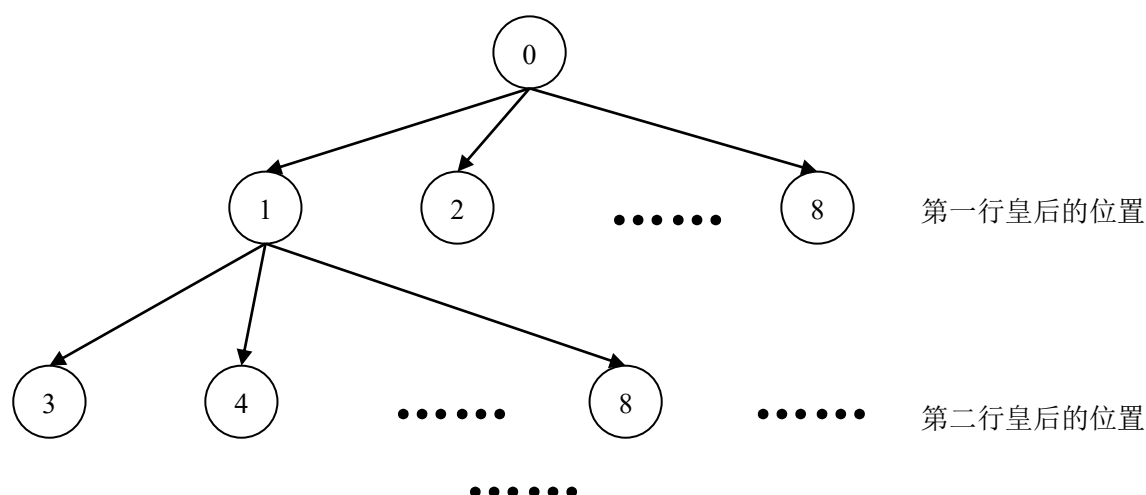
N 皇后问题的递归算法的程序如下：

```
program N_Queens;
const
    MaxN = 100;                {最多皇后数}
var
    A:array [1..MaxN] of Boolean;    {竖线被控制标记}
    B:array [2..MaxN * 2] of Boolean; {左上到右下斜线被控制标记}
    C:array [1 - MaxN..MaxN - 1] of Boolean; {左下到右上斜线被控制标记}
    X: array [1 .. MaxN] of Integer; {记录皇后的解}
    Total: Longint;               {解的总数}
    N: Integer;                   {皇后个数}

procedure Out;                   {输出方案}
var
    I: Integer;
begin
    Inc(Total); Write(Total: 3, ' ');
    for I := 1 to N do Write(X[I]: 3);
    Writeln;
end;

procedure Try(I: Integer);
{搜索第 I 个（行）的皇后的可行位置}
var
    J: Integer;
begin
    if I = N + 1 then Out; {边界条件，N 个皇后都放置完毕，则输出解}
    for J := 1 to N do {第 I 个皇后从第 1 列到第 N 列}
        if A[J] and B[J + I] and C[J - I] then
            begin
                X[I] := J; ; {置第 i 个皇后的位置是第 j 列}
                A[J] := False; ; {宣布占领行、对角线}
                B[J + I] := False;
                C[J - I] := False;
                Try(I + 1); {搜索下一皇后的位置}
                A[J] := True; ; {撤消被占行、对角线，回溯}
                B[J + I] := True;
                C[J - I] := True;
            end;
    end;
end;
begin
    Write('Queens Numbers = ');
    Readln(N);
    FillChar(A, Sizeof(A), True);
    FillChar(B, Sizeof(B), True);
    FillChar(C, Sizeof(C), True);
    Try(1);
    Writeln('Total = ', Total);
end.
```

从数据结构的观点来看回溯法，其实就是一棵搜索树的前序遍历。



前序遍历算法递归伪代码：

```
Procedure try(i:Node);访问第节点 i
Begin
    递归边界;
    访问节点 I;
    依次访问节点 I 的所有儿子;
End;
```

前序遍历的特点：按前序遍历访问一棵树得到的节点访问序列中，每个节点的儿子在它的右边。
上图的访问序列为：0134828

★ 小结：递归回溯通用算法(背住)

try(i) 第 i 步(深度)；

var

x:integer; //思考：为什么 x 只是能局部变量？

begin

if i=maxi then 输出并返回 //递归边界

for x:=1 to max //共有 max 种走法，每一种都要试一试。

begin

if 没有冲突 then //判断当前第 x 种走法是否可行

begin

记录方案

宣布占领

try(i+1)

宣布释放（回溯）{以便试第 x+1 种走法（儿子 A 返回后再访问下一个儿子 B 之前要把儿子 A 作的修改撤消）}

end;

end;

end;

回溯法的关键在于：儿子节点在返回到父节点时（回溯）需要把它作的修改撤消，以便父节点访问下一个节点。